

Let's look into the OpenLDAP schema files to determine which object classes contain the corresponding attributes. Since the name of a person is designated as the 'Common Name' or the 'cn' attribute, a search on an objectclass containing the 'cn' attribute would be ideal. Let us search the `core.schema` file for such an objectclass:

```
[root@vbg schema]# cat /etc/openldap/schema/core.schema | egrep -A 5
^objectclass | grep -B 3 cn | head -n 5
objectclass { 2.5.6.6 NAME 'person'
    DESC 'RFC256: a person'
    SUP top STRUCTURAL
    MUST ( sn $ cn )
```

I have piped the output to 'head' to show the first objectclass that contains the attribute 'cn' (short for *commonName*). From the above output, it seems that we might use the objectclass 'person' in our address book. Let's see what other attributes this objectclass provides:

```
[root@vbg schema]# cat /etc/openldap/schema/core.schema | grep -A 5
^objectclass { 2.5.6.6 NAME 'person'
objectclass { 2.5.6.6 NAME 'person'
    DESC 'RFC256: a person'
    SUP top STRUCTURAL
    MUST ( sn $ cn )
    MAY ( userPassword $ telephoneNumber $ seeAlso $ description ) )
```

We find that the *person* objectclass has the attributes - *sn* (surname), *cn* (common name), *userPassword*, *telephoneNumber*, *seeAlso* and *description*. From this, it's clear that the *person* objectclass alone will not suffice for our address book, since it does not have attributes for the person's address, mobile number and e-mail address.

Before continuing our search for other objectclasses that will provide us with the other required attributes, let's take a look at the *objectclass* definition syntax. The exact formal syntax is defined in RFC 2252, but I present a simplified syntax below:

```
objectclass (<OID> NAME <name of the objectclass>
    DESC <description>
    SUP <name of the parent objectclass; similar to attributes
explained in the last article> <type of this objectclass>
    MUST <attributes that must be given a value when using this
class>
    MAY <attributes that may be given a value, or left empty>))
```

Looking at the definition of the *person* objectclass, and matching it with the syntax definition provided above, we can interpret the *person* objectclass definition as follows:

- *OID* (unique object ID for the objectclass) = 2.5.6.6
- *NAME* of the objectclass = 'person'
- Superior/parent objectclass = 'top'
- Type of the *person* objectclass = *STRUCTURAL MUST* (mandatory attributes that cannot be left empty) = *sn, cn*

- *MAY* (optional attributes) = *userPassword*, *telephoneNumber*, *seeAlso* and *description*

## Types of objectclasses

There are three types of objectclasses: abstract, auxiliary and structural.

A structural objectclass provides the underlying schema for a record; every entry/record in a directory needs to have a structural objectclass. You can use only one structural class in an entry, i.e., you cannot define an entry with two structural classes. There is an exception though: you can specify multiple structural classes that are part of the same objectclass hierarchy (the parent-child relationship).

Objectclasses, like attributes, as discussed in the previous article, have a hierarchical relationship. A child objectclass inherits the attributes of its parents. You don't really need to specify a given structural objectclass in an entry if you specify one of its children; the attributes that you might want to use from the parent objectclass would already be available, having been inherited from the parent objectclass.

Given below is an example of an invalid entry:

```
objectclass: person
objectclass: organizationalRole
```

If you look into the *core.schema* file and check these objectclass definitions, you will see that both these objectclasses are structural, and both have *top* as their superior—they are not related to each other. An entry with the following object classes, though, is valid:

```
objectclass: person
objectclass: organizationalPerson
```

Although both these classes are structural, the definition for the *organizationalPerson* objectclass specifies that the *person* objectclass is its superior:

```
[root@vbg schema]# cat /etc/openldap/schema/core.schema | grep -A 2
^objectclass { 2.5.6.7 NAME 'organizationalPerson'
objectclass { 2.5.6.7 NAME 'organizationalPerson'
    DESC 'RFC256: an organizational person'
    SUP person STRUCTURAL
```

Auxiliary classes can be attached to any entry that has a structural class. They are used to provide extra attributes for an entry, without altering the basic structure of the entry. This is very useful in advanced LDAP usage. As an example, the objectclass *pkiUser* can be used to represent certificate objects:

```
[root@vbg schema]# cat /etc/openldap/schema/core.schema | grep -A 5
^objectclass { 2.5.6.21 NAME 'pkiUser'
objectclass { 2.5.6.21 NAME 'pkiUser'
    DESC 'RFC2587: a PKI user'
```